



**COMSATS University Islamabad,
Sahiwal Campus.**

SWIFT QUEUE

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Areesh Fatima

FA22-BCS-022

Laiba Naeem

FA22-BCS-042

Arham Butt

FA22-BCS-060



**COMSATS University Islamabad,
Sahiwal Campus.**

SWIFT QUEUE

By

Areesh Fatima

FA22-BCS-022

Laiba Naeem

FA22-BCS-042

Arham Butt

FA22-BCS-060

Supervisor

Mr. Muhammad Umar

Bachelor of Science in Computer Science (2022-2026)

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Areesh Fatima

Laiba Naeem

Arham Butt

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) “SWIFT QUEUE” was developed by **Areesh Fatima (FA22-BCS-022)**, **Laiba Naeem (FA22-BCS-042)** and **Arham Butt (FA22-BCS-060)** under the supervision of “**Mr. Muhammad Umar**” and co-supervisor “**Mr. Ahmad Farid**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor

Mr. Muhammad Umar

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

In today's fast-paced world, the healthcare sector faces significant challenges in managing patient flow efficiently, particularly in hospitals and clinics where long waiting times and overcrowding are common. Traditional queue management systems rely heavily on manual processes, resulting in delays, lack of transparency, and poor patient experience. SwiftQueue is developed as an innovative solution to address these issues by introducing a smart, digital queue management system that enhances efficiency, convenience, and accessibility for both patients and healthcare providers.

SwiftQueue consists of two main components: a mobile application for patients and a web-based admin panel for doctors and administrators. The mobile application allows patients to join queues remotely using QR code technology, eliminating the need for physical waiting. It provides real-time queue tracking, enabling users to monitor their position and receive timely notifications when their turn approaches. This feature significantly reduces uncertainty and allows patients to manage their time more effectively. Additionally, the application includes appointment booking functionality, which helps minimize congestion and improve scheduling efficiency.

On the other hand, the web-based admin panel is designed to provide centralized control to doctors and administrators. Through this panel, administrators can manage doctor registrations by approving verified doctors, maintain records of all registered doctors, and handle patient complaints efficiently. Doctors can also manage queues, update patient status, and monitor system activity in real time. This integration ensures smooth coordination between patients and healthcare providers.

A key feature of SwiftQueue is its integration of location-based services through map functionality. Patients can view the exact location of clinics and calculate travel time using GPS and mapping APIs. This ensures that users can plan their arrival efficiently, reducing delays and improving overall coordination. The inclusion of map integration enhances the system's usability by combining navigation with queue management.

The system is developed using modern technologies such as Flutter for mobile application development and Firebase for backend services. Firebase Authentication ensures secure user login, while Firestore provides real-time data synchronization, allowing instant updates across both the mobile application and web system. Push notification services are implemented to deliver alerts and reminders, ensuring effective communication between the system and its users.

Acknowledgment

All praise is to almighty Allah who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Mr. Muhammad Umar**” and our Co-Supervisor “**Mr. Ahmad Farid**”. Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

Areesh Fatima

Laiba Naeem

Arham Butt

Abbreviations

API	Application Programming Interface
QR	Quick Response Code
UI	User Interface
Auth	Authentication
ETA	Estimated Time of Arrival
OPD	Outpatient Department
RTQS	Real-Time Queue system

Table of Contents

Chapter 1 Introduction

1	Introduction.....	1
1.1	Brief Overview.....	2
1.2	Relevance to Course Modules.....	2
1.3	Project Background.....	3
1.4	Literature Review.....	4
1.5	Analysis from Literature Review.....	5
1.6	Methodology and Software Lifecycle.....	6
1.6.1	Rationale behind Selected Agile Model.....	7

Chapter 2 Problem Definition

2	Problem Definition.....	10
2.1	Problem Statement.....	10
2.2	Deliverables and Development Requirements.....	10

Chapter 3 Requirement Analysis

3	Requirement Analysis.....	11
3.1	Use Case Diagram.....	11
3.2	Use Case Descriptions.....	12
3.3	Functional Requirements.....	19
3.3.1	User Registration & Authentication.....	19
3.3.2	Doctor Profile Management.....	19
3.3.3	QR Code Generation & Scanning.....	19
3.3.4	Queue Joining System.....	19
3.3.5	Appointment Slot Booking.....	19
3.3.6	Real-Time Queue Tracking.....	20
3.3.7	Notifications & Alerts.....	20
3.3.8	Patient History Management.....	20
3.3.9	Location & Map Integration.....	20
3.3.10	Admin Pannel Management.....	20
3.3.11	Doctor verification system.....	20

3.3.12 Complaint & Report Management.....	20
3.4 Nonfunctional Requirements.....	20
3.4.1 Performance.....	21
3.4.2 Security.....	21
3.4.3 Scalability.....	21
3.4.4 Accessibility.....	21
3.4.5 Usability.....	21
Chapter 4 Design and Architecture	
4 Design and Architecture.....	22
4.1 System Architecture.....	23
4.2 Process Flow/Representation.....	24
4.3 Sequence Diagram.....	26
4.4 Class Diagram.....	29
Chapter 5 Implementation	
5 Implementation.....	31
5.1 User Registration Algorithm.....	31
5.2 QR Code Generation Algorithm.....	31
5.3 Queue Management Algorithm.....	31
5.4 Appointment Scheduling Logic.....	32
5.5 Security and Data Protection.....	32
5.6 Real-Time Queue Tracking System.....	32
5.7 User Interface.....	32
5.7.1 Welcome Screen.....	33
5.7.2 Login / Signup.....	33
5.7.3 Patient Dashboard.....	34
5.7.4 Doctor Dashboard.....	35
5.7.5 QR Scan Screen.....	35
5.7.6 Queue Status Screen.....	36
5.8 Smart Queue Estimation System.....	36

Chapter 6 Testing

6	Testing.....	38
6.1	Unit Testing.....	38
6.2	Manual Testing.....	38
6.3	Functional Testing.....	38
6.4	Integration Testing.....	39
6.5	System Testing.....	39
6.6	Testing Summary.....	40

Chapter 7 Conclusion

7	Conclusion and Future Work.....	42
7.1	Conclusion.....	42
7.2	Future Work.....	42
7.2.1	Real-time Queue Prediction.....	42
7.2.2	Multi-Hospital Integration.....	42
7.2.3	Xanp Firebase Integration.....	42
7.3	Limitations of the system.....	42
	References.....	43

List of Figures

Figure 3.1 Use Case Diagram	11
Figure 4.1 System Architecture	24
Figure 4.2 Queue Flowchart	25
Figure 4.3 Sequence Diagram	26
Figure 4.4 Class Diagram	29
Figure 4.5 ER Diagram.....	29
Figure 5.8 Welcome Screen	33
Figure 5.9 Login Page	33
Figure 5.10 Patient Dashboard	34
Figure 5.11 Doctor Dashboard.....	35
Figure 5.12 QR scan Screen	35
Figure 5.13 Queue Status Screen.....	36

List of Tables

Table 1.1 Comparison with Existing Systems	4
Table 3.1 Use Case Description for Join Queue.....	12
Table 3.2 Use Case Description for Register/Login	12
Table 3.3 Use Case Description for Book Appointment	13
Table 3.4 Use Case Description for Scan QR Code.....	14
Table 3.5 Use Case Description for Receive Notifications	14
Table 3.6 Use Case Description for View map Location.....	14
Table 3.7 Use Case Description for Queue Tracking.....	15
Table 3.8 Use Case Description for Viewing History.....	15
Table 3.9 Use Case Description for Generating QR Code.....	16
Table 3.10 Use Case Description for Admin Login.....	17
Table 3.11 Use Case Description for Viewing Patients.....	17
Table 3.12 Use Case Description for Handle Patient Complain.....	17
Table 3.13 Use Case Description for Managing Queue.....	18
Table 3.14 Use Case Description for Approving Doctors.....	18
Table 6.1 Unit Testing	38
Table 6.2 Manual Testing	38
Table 6.3 Functional Testing	38
Table 6.4 Integration Testing	39
Table 6.5 System Testing	40

Chapter 1

Introduction

1. Introduction

The rapid advancement of information technology has transformed almost every sector, including healthcare. Hospitals and clinics are continuously striving to improve the quality of their services while minimizing inefficiencies. One of the most critical challenges faced by healthcare institutions is the management of patient queues. Traditional queue management systems rely heavily on manual processes, where patients are required to wait physically in long lines without clear information about their waiting time or turn. This often leads to overcrowded waiting areas, increased stress among patients, and inefficient use of hospital resources.

In developing countries, these issues are even more prominent due to high patient volumes and limited infrastructure. Patients frequently spend long hours waiting for consultation, which negatively affects their experience and reduces trust in healthcare services. At the same time, healthcare staff face difficulties in managing queues manually, resulting in errors, delays, and poor coordination.

SwiftQueue is proposed as an innovative solution to these challenges. It is a smart digital queue management system that combines a mobile application for patients and a web-based admin panel for doctors and administrators. The mobile application enables patients to join queues remotely using QR code technology, track their position in real time, and receive notifications when their turn approaches. This eliminates the need for physical waiting and allows users to manage their time more efficiently.

The web-based admin panel provides centralized control to healthcare providers. Through this system, administrators can approve doctor registrations, manage doctor profiles, monitor patient queues, and handle patient complaints. Doctors can also update queue status and manage patient flow effectively, ensuring smooth operation of the system.

The application is developed using modern technologies such as Flutter for mobile development and Firebase for backend services. These technologies enable secure authentication, real-time data synchronization, and scalable performance. By integrating both mobile and web platforms, SwiftQueue creates a complete ecosystem that improves coordination between patients and healthcare providers.

Overall, SwiftQueue enhances efficiency, reduces waiting time, and improves the overall healthcare experience by introducing a reliable and user-friendly digital solution for queue management.

1.1 Brief Overview

SwiftQueue is a comprehensive digital system designed to revolutionize the way patient queues are managed in healthcare environments. The system consists of a mobile application for patients **and** a web-based admin panel for doctors and administrators, providing a unified platform where users can interact efficiently without the limitations of traditional queue systems.

The core idea behind SwiftQueue is to replace manual queue management with an automated system that ensures transparency, efficiency, and convenience. Patients can create accounts, log in securely, and scan QR codes provided by doctors or clinics to join queues instantly through the mobile application. Once added to the queue, the system assigns a position and continuously updates it based on real-time changes. This allows patients to track their status and manage their time effectively.

Doctors and administrators are provided with a dedicated web-based dashboard where they can monitor queues, manage patient flow, approve doctor registrations, and update their availability. This ensures that the consultation process remains organized and predictable. Additionally, the system includes features such as appointment scheduling, patient history management, and notifications, making it a complete healthcare management solution.

The use of Firebase enables real-time updates, ensuring that all users receive accurate information instantly across both mobile and web platforms. The system is designed with a user-friendly interface, making it accessible to users with varying levels of technical expertise.

1.2 Relevance to Course Modules

The SwiftQueue project is highly relevant to the academic curriculum of Software Engineering and integrates multiple course modules into a single practical application involving both a mobile application and a web-based admin panel. Firstly, the project demonstrates the application of Software Engineering principles, including requirement analysis, system design, development methodologies, and testing strategies. The Agile methodology used in this project reflects modern industry practices, emphasizing iterative development and continuous improvement across both mobile and web components. This approach ensures flexibility, faster development cycles, and better adaptation to changing requirements throughout the project lifecycle.

Secondly, the project incorporates key concepts from Database Management Systems. Firebase Firestore is used as a NoSQL database to store and manage user data, queue information, appointment details, and doctor records. The database design ensures efficient data retrieval, real-time updates, and scalability, which are essential for handling multiple users simultaneously. The integration of cloud-based database services enhances system performance and reliability while supporting seamless synchronization between different components of the system.

Thirdly, Mobile Application Development plays a central role in this project. Flutter is used to develop a responsive and visually appealing user interface for patients. The mobile application provides features such as QR code scanning, real-time queue tracking, notifications, and appointment booking. At the same time, the web-based admin panel supports system control and management operations, enabling administrators and doctors to manage queues, approve users, and monitor activities effectively.

Furthermore, Networking concepts are applied through real-time data communication between the mobile application, web panel, and backend services. Firebase ensures seamless synchronization, allowing users and administrators to receive updates instantly. This real-time interaction improves system efficiency and enhances user experience by minimizing delays and providing accurate information at all times.

The project also reflects principles of Human-Computer Interaction (HCI) by focusing on usability and accessibility. The interfaces are designed to be intuitive and user-friendly, reducing the learning curve for patients, doctors, and administrators. Proper layout design, navigation structure, and visual elements ensure that users can interact with the system easily and efficiently.

Lastly, Project Management concepts are implemented through proper planning, task division, and timeline management. The project was developed using a structured approach, ensuring coordination between team members and successful integration of both mobile and web components. This systematic management contributed to the timely completion and overall success of the SwiftQueue system.

1.3 Project Background

The idea of SwiftQueue emerged from observing real-world problems in healthcare facilities, where inefficient queue management leads to dissatisfaction among both patients and medical staff. In many hospitals and clinics, patients are required to take physical tokens and wait in crowded areas for extended periods of time. This traditional approach not only wastes valuable time but also creates an uncomfortable and stressful environment, especially for elderly patients, children, and individuals with medical conditions. Long waiting times and lack of proper organization often result in frustration and reduced quality of service.

Existing solutions, such as appointment booking systems, attempt to address some aspects of the problem but still fall short in providing a complete solution. While these systems allow patients to reserve time slots in advance, they do not offer real-time updates or dynamic queue adjustments. As a result, patients may miss their turn due to delays, lack of timely notifications, or sudden changes in schedule. On the other hand, doctors and healthcare staff face difficulties in managing unpredictable patient flow, leading to inefficiencies in daily operations and increased workload pressure.

With the rapid growth in smartphone usage and internet connectivity, there is a significant opportunity to transform traditional healthcare systems into more efficient digital solutions. SwiftQueue leverages this opportunity by introducing a smart, mobile-based queue management system that integrates modern technologies such as QR codes, cloud databases, and real-time tracking. By using QR codes, patients can easily join queues without the need for physical tokens, while real-time updates ensure that they are informed about their position and estimated waiting time.

The system also provides seamless communication between patients, doctors, and administrators, reducing confusion and improving coordination. The use of cloud-based technologies such as Firebase enables secure data storage, fast data retrieval, and instant synchronization across multiple devices. This ensures that all users receive accurate and up-to-date information at all times, regardless of their location.

The primary aim of this project is to modernize healthcare queue management by providing a system that is efficient, reliable, and user-friendly. SwiftQueue not only minimizes waiting time but also enhances the overall patient experience by offering convenience and transparency. Furthermore, it helps healthcare providers manage their operations more effectively by organizing patient flow and reducing manual workload.

In addition, SwiftQueue sets a strong foundation for future advancements in digital healthcare systems. It can be extended with features such as AI-based queue prediction, multi-hospital integration, and advanced analytics for performance monitoring. By combining innovation with practical usability, the project contributes towards building smarter and more accessible healthcare solutions for the future.

1.4 Literature Review

A comprehensive literature review was conducted to understand existing systems and identify gaps in current solutions. Various queue management systems and healthcare applications were analyzed, including hospital management software, appointment booking platforms, digital token systems, and web-based administrative solutions. These systems were carefully studied to evaluate their strengths, weaknesses, and overall effectiveness in handling patient flow and service delivery.

Traditional hospital systems primarily rely on manual processes or basic digital tools that do not offer real-time tracking or centralized management. Appointment booking systems allow users to reserve time slots, but they do not adapt to delays or changes in real-time. Similarly, many web-based systems focus only on administrative control and lack proper integration with patient-side mobile applications, resulting in inefficiencies and poor user experience. This lack of coordination often leads to confusion, longer waiting times, and reduced service quality.

Some modern applications provide queue management features, but they lack full integration between mobile platforms and web-based admin panels and do not utilize advanced technologies such as QR codes, cloud computing, and real-time synchronization. Additionally, many systems are expensive and not accessible to smaller healthcare facilities. This creates a gap where affordable, scalable, and integrated solutions are still needed.

Research studies highlight the importance of real-time systems in improving efficiency and reducing waiting times. The integration of mobile applications with cloud-based services and web systems has been proven to enhance user experience, coordination, and system performance. These findings emphasize the need for smart, technology-driven solutions in modern healthcare environments.

SwiftQueue builds upon these findings by combining multiple technologies into a single platform that includes both a mobile application and a web-based admin panel. It addresses the limitations of existing systems by providing real-time updates, dynamic queue management, centralized control, and user-friendly interfaces. Furthermore, the system ensures better

communication between patients and healthcare providers, ultimately improving overall efficiency and satisfaction.

Table 1.1: Comparison with Existing Systems

Existing System	Drawbacks	Proposed System Solutions (SwiftQueue)
Manual Queue System	Long waiting times and overcrowding	Real-time digital queue tracking through mobile app
	No transparency in queue status	Live queue position updates for patients
Appointment Systems	Fixed schedules without real-time updates	Dynamic queue system with real-time adjustments
	Poor handling of delays and emergencies	Instant notifications for users
Traditional Hospital Management	Manual record handling and inefficiency	Automated queue management system
	Difficult to manage large number of patients	Centralized system with admin control
Basic Healthcare Apps	Lack of admin monitoring features	Web-based admin panel using Laravel
	Limited functionality and integration	Integration with Firebase for real-time updates
Systems without Navigation	Patients face difficulty locating clinics	Google Maps integration for location and routing

The table describes the key differences between traditional healthcare queue systems and the proposed SwiftQueue system. It explains problems such as long waiting times, lack of transparency, and manual queue handling in existing systems. The table also highlights advanced features of SwiftQueue including real-time queue tracking, instant notifications, and automated management. Furthermore, it shows the use of technologies like Firebase, Laravel, and Google Maps integration to improve system efficiency. Overall, the table demonstrates how SwiftQueue provides a modern and user-friendly healthcare queue management solution.

1.5 Analysis from Literature Review

After conducting a detailed literature review of existing healthcare queue management systems and related technologies, several important insights were identified. These insights highlight both the strengths and limitations of current systems and justify the need for the proposed solution, SwiftQueue.

Many traditional queue management systems rely on manual processes or static appointment schedules. These systems often result in long waiting times, overcrowding in clinics, and lack of transparency for patients. Patients are usually unaware of their exact position in the queue, which leads to frustration and inefficient time management.

Modern digital solutions have attempted to address these issues by introducing appointment-based systems and basic queue tracking. However, these systems still face limitations such as lack of real-time updates, limited user interaction, and absence of location-based guidance. Additionally, many existing applications do not provide integrated features such as QR-based check-in, live queue updates, and dynamic time estimation.

The literature review also reveals that technologies like mobile applications, cloud databases, and real-time data synchronization can significantly improve system efficiency. Firebase-based systems, for example, enable real-time updates and seamless communication between users and the backend. Similarly, QR code technology simplifies user interaction by allowing quick and secure access to queue systems.

Based on these findings, SwiftQueue is designed to overcome the identified limitations by integrating modern technologies into a unified platform. The system provides real-time queue tracking, QR-based queue joining, location assistance using map services, and a centralized admin panel for managing doctors and patients. This ensures improved efficiency, transparency, and user satisfaction.

In conclusion, the analysis of existing systems clearly indicates a gap in providing a fully integrated, real-time, and user-friendly healthcare queue management system. SwiftQueue addresses this gap by offering a comprehensive solution that enhances both patient experience and administrative control.

1.6 Methodology and Software Lifecycle

The development of SwiftQueue follows the Agile software development methodology, which is widely used in modern software engineering. Agile emphasizes iterative development, collaboration, and adaptability, making it highly suitable for projects with evolving requirements and continuous improvement. In this project, Agile was applied to develop both

the mobile application and the web-based admin panel in a coordinated and efficient manner. The use of Agile allowed the development team to respond quickly to changes, incorporate feedback, and ensure that both system components remained aligned throughout the development process.

The project lifecycle is divided into several phases, including requirement gathering, system design, implementation, testing, and deployment. During the requirement gathering phase, detailed analysis was conducted to identify the needs of different users, including patients, doctors, and administrators. Features for both the mobile application (patient side) and the web system (admin/doctor side) were carefully defined. The mobile application focused on functionalities such as QR code scanning, real-time queue tracking, notifications, appointment booking, and map integration for location services. On the other hand, the web panel was designed to handle administrative tasks such as doctor management, queue monitoring, patient record handling, and complaint management.

In the system design phase, the architecture of the application was planned, including database design, user interface layouts, and interaction between different modules. Firebase Firestore was selected as the cloud-based database due to its real-time synchronization capabilities and scalability. The system design ensured smooth communication between the mobile application and the web-based admin panel, enabling seamless data flow and efficient performance.

During the implementation phase, development was carried out in short iterations known as sprints. Each sprint focused on delivering specific features of the system. Initially, core functionalities such as user authentication, registration, and basic queue management were implemented. Gradually, more advanced features such as real-time updates, push notifications, QR code integration, and map services were added. Both the mobile application and the web system were developed in parallel, ensuring proper integration and synchronization of data across both platforms.

Testing was performed continuously after each iteration to ensure system reliability and performance. Individual features were tested independently (unit testing) and later integrated to verify smooth communication between the mobile application and the web panel (integration testing). Functional testing was also conducted to ensure that each feature performed according to the defined requirements. Regular feedback was collected from users and supervisors, which helped in identifying issues and improving the overall system usability and functionality. The deployment phase involved making the system available for use after thorough testing and validation. The application was configured to work efficiently with cloud services, ensuring

that real-time data updates and synchronization were maintained. Proper attention was given to performance optimization, security, and user experience during this phase.

Overall, the Agile approach enabled efficient development, flexibility in handling changes, and successful integration of both the mobile and web components. This approach significantly enhanced the quality of the final system and ensured that it meets user requirements effectively. Additionally, it allowed continuous improvement throughout the development process, resulting in a robust, scalable, and user-friendly healthcare queue management system.

1.6.1 Rationale behind Selected Agile Model

The Agile model was chosen for the development of SwiftQueue due to its flexibility and strong ability to accommodate changing requirements throughout the software development process. Unlike traditional development models that follow a rigid and sequential structure, Agile allows continuous adaptation and improvement. Since SwiftQueue includes both a mobile application and a web-based admin panel, the system requirements were not completely fixed at the beginning and evolved during development. Agile provided the development team with the ability to respond quickly to these changes, ensuring that both components were developed in alignment and integrated smoothly.

One of the primary reasons for selecting Agile was its iterative nature, which allows the project to be developed in small, manageable increments known as sprints. Each sprint focuses on delivering a functional part of the system, which can be tested and improved before moving on to the next phase. This approach reduced the risk of major errors and allowed early detection of issues. It also ensured that the core features of SwiftQueue, such as user authentication, queue management, and real-time tracking, were implemented and refined step by step.

Another important advantage of Agile is enhanced collaboration among team members. Since SwiftQueue involves multiple components, including frontend mobile development, backend integration, and web-based administration, effective communication between team members was essential. Agile promotes teamwork through regular meetings, discussions, and progress reviews, ensuring that all members stay aligned with the project goals. This collaborative environment helped in efficiently managing both mobile and web modules without conflicts or delays.

Agile also supports continuous testing and feedback, which played a significant role in improving the quality of the system. After each iteration, features were tested and evaluated to ensure they met the desired requirements. Feedback from users and supervisors was incorporated into subsequent iterations, allowing the system to evolve according to real-world

needs. This continuous improvement process enhanced system reliability and usability, making SwiftQueue more effective and user-friendly.

Key advantages of Agile in this project include continuous delivery of functional modules for both mobile and web systems, improved collaboration among team members working on different components, faster identification and resolution of issues during development, and enhanced user satisfaction through regular feedback and testing. These benefits contributed to the successful development of a robust and scalable system.

In addition, Agile allowed better time management and project control by breaking down complex tasks into smaller units. This made it easier to monitor progress, meet deadlines, and ensure that each feature was properly implemented. The flexibility of Agile also ensured that new features, such as QR code integration and real-time notifications, could be added without disrupting the overall system structure.

Overall, the selection of the Agile model proved to be highly effective for the SwiftQueue project. It provided the necessary flexibility, improved communication, and ensured continuous improvement throughout the development lifecycle. As a result, the final system meets user requirements efficiently while maintaining high performance, reliability, and scalability.

Chapter 2

Problem Definition

2.1 Problem Statement

Hospitals and clinics face significant challenges in managing patient queues efficiently, particularly in environments where traditional manual systems are still in use. In many healthcare facilities, patients are required to physically visit the hospital, take a token, and wait for their turn in crowded waiting areas. This process often leads to long waiting times, overcrowding, and an overall lack of organization. Such inefficiencies not only waste valuable time for patients but also create a stressful and uncomfortable environment, especially for individuals who are already unwell or require urgent medical attention.

One of the major issues with manual queue management systems is the lack of transparency. Patients are usually unaware of their exact position in the queue or the estimated time they will be attended. This uncertainty often results in frustration, dissatisfaction, and repeated inquiries at reception desks, further increasing the workload on hospital staff. Additionally, there is no proper mechanism to handle unexpected delays, such as emergencies or doctor unavailability, which can disrupt the entire queue system and lead to confusion among patients.

From the administrative perspective, managing patient flow and doctor availability becomes a complex and challenging task. Administrators often rely on basic tools or manual tracking methods, which are prone to errors and inefficiencies. Coordinating between multiple doctors, handling walk-in patients, and managing appointments simultaneously can lead to mismanagement and delays in service delivery. Furthermore, the absence of centralized control makes it difficult to monitor real-time queue status and make informed decisions.

Although some digital solutions such as appointment booking systems exist, they only partially address the problem. These systems allow patients to reserve time slots in advance but do not provide real-time updates or dynamic queue adjustments. As a result, patients may still experience delays, miss their turn, or face scheduling conflicts. Moreover, many existing systems lack proper integration between patient-side applications and administrative control panels, leading to fragmented and inefficient operations.

With the increasing availability of smartphones and internet connectivity, there is a growing need for a modern solution that can automate and optimize the queue management process. Such a system should not only allow patients to join queues remotely but also provide real-time updates regarding their position and estimated waiting time. At the same time, it should enable administrators to manage queues, monitor doctor availability, and handle patient data efficiently through a centralized platform.

SwiftQueue addresses these challenges by introducing a comprehensive digital platform that combines a mobile application for patients with a web-based admin panel for doctors and administrators. The system leverages technologies such as QR codes, cloud databases, and real-time synchronization to provide an efficient and user-friendly solution. Patients can join queues remotely, receive instant notifications, and track their position in real time, while administrators can manage the entire system through a centralized interface.

Overall, the SwiftQueue system aims to enhance efficiency, improve coordination, and provide a seamless user experience for both patients and healthcare providers. By replacing traditional manual processes with an intelligent digital solution, the system significantly reduces waiting times, minimizes confusion, and improves the overall quality of healthcare services.

2.2 Deliverables and Development Requirements

The primary deliverable of this project is a fully functional system consisting of a mobile application for patients and a web-based admin panel for doctors and administrators, enabling digital queue management.

Key deliverables include:

- Patient and doctor dashboards (mobile and web)
- QR-based queue system
- Real-time queue tracking
- Secure authentication system
- Admin panel for doctor approval and complaint handling.

Development requirements include:

- Flutter framework for mobile frontend
- Web technologies for admin panel development
- Firebase for backend services
- Cloud Firestore database
- Secure authentication mechanisms.

Chapter 3

Requirement Analysis

3.1 Use Case Diagram

The system includes three main actors:

- Patient
- Doctor
- Admin

Patients can register, scan QR codes, join queues, track status, and view locations using the mobile application. Doctors and administrators use the web-based admin panel to manage queues, approve doctors, and monitor system activities.

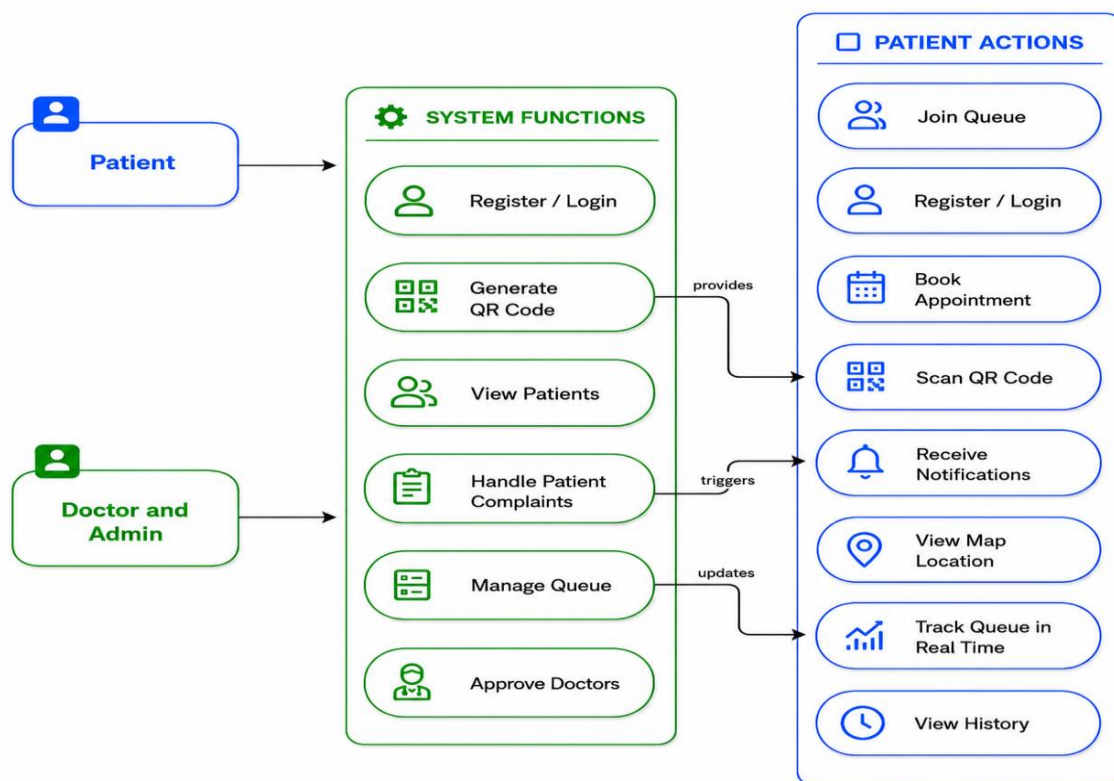


Figure 3.1

This diagram shows the interaction between patients, doctors/admins, and the SwiftQueue system functionalities. Patients can perform actions such as registration/login, joining queues, booking appointments, scanning QR codes, and tracking queues in real time. Doctors and admins manage queues, approve doctors, generate QR codes, handle complaints, and view patient records. The arrows represent the communication and data flow between administrative operations and patient activities within the system. Overall, the diagram provides a clear overview of the integrated workflow and real-time healthcare queue management process of SwiftQueue.

3.2 Use Case Descriptions

Each use case defines:

- Preconditions
- Actions
- Outcomes

Examples include login, QR scanning, queue joining, appointment booking, doctor approval, and complaint handling, covering functionalities of both the mobile application and the web-based admin panel.

Table 3.1: Join Queue

Field	Description
Test Case Id	UC-07
Test Case Name	Join Queue
Actor	Patient
Objective	To allow patient to join doctor's queue
Module	Mobile App
Input	Click "Join Queue"
Expected Result	Patient added to queue
Actual Result	As expected
Status	Pass

Table 3.1 describes the “Join Queue” use case of the SwiftQueue system. This use case allows a patient to join a doctor’s queue through the mobile application. The table explains the objective, module, input, and expected outcome of the process. The patient selects the “Join Queue” option to enter the active queue of a doctor. The expected result is that the patient is successfully added to the queue with a queue position assigned. The actual result shows that the functionality worked as expected during testing. The status of the test case is marked as “Pass,” indicating successful execution. This use case is an important feature of SwiftQueue because it enables digital queue management and reduces physical waiting time. Overall, the table demonstrates the successful implementation and testing of the queue joining functionality in the system.

Table 3.2: Register / Login

Field	Description
Test Case Id	UC-08
Test Case Name	Register / Login
Actor	Doctor/Admin/Patient
Objective	To authenticate user access
Module	Mobile App
Input	Email and Password
Expected Result	User logged in successfully
Actual Result	As expected
Status	Pass

Table 3.2 describes the “Register / Login” use case of the SwiftQueue system. This use case is responsible for authenticating users and allowing secure access to the mobile application. The table explains that the user enters an email and password as input to access the system. The main objective of this functionality is to verify user credentials and provide authorized access to system features. The expected result is that the user logs in successfully after entering valid information. The actual result shows that the system performed as expected during testing. The status of the test case is marked as “Pass,” which indicates successful execution of the login process. This functionality is important because it ensures system security and protects user information. Overall, the table demonstrates the successful implementation and testing of the registration and login feature in SwiftQueue.

Table 3.3: Book Appointment

Field	Description
Test Case Id	UC-09
Test Case Name	Book Appointment
Actor	Patient
Objective	To allow patient to book appointment
Module	Mobile App
Input	Select date & time
Expected Result	Appointment confirmed
Actual Result	As expected
Status	Pass

Table 3.3 describes the “Book Appointment” use case of the SwiftQueue system. This use case allows a patient to book an appointment through the mobile application by selecting a suitable date and time. The table explains the objective, module, input, and expected outcome of the appointment booking process. The expected result is that the appointment is successfully confirmed after the patient provides the required details. The actual result indicates that the system performed as expected during testing. The status of the test case is marked as “Pass,” showing that the functionality was executed successfully. This feature helps patients schedule appointments conveniently without visiting the clinic physically. It also improves time management for both doctors and patients through digital appointment handling. Overall, the table demonstrates the successful implementation and testing of the appointment booking functionality in SwiftQueue.

Table 3.4: Scan QR Code

Field	Description
Test Case Id	UC-10
Test Case Name	Scan QR Code
Actor	Patient
Objective	To join queue via QR
Module	Mobile App
Input	Scan QR code
Expected Result	Queue joined successfully
Actual Result	As expected
Status	Pass

Table 3.4 describes the “Scan QR Code” use case of the SwiftQueue system. This use case allows a patient to join a doctor’s queue by scanning a QR code through the mobile application. The table explains the objective, module, input, and expected outcome of the QR scanning process. The patient scans the QR code provided by the doctor to access and join the queue instantly. The expected result is that the patient successfully joins the queue after scanning the code. The actual result indicates that the system performed as expected during testing. The status of the test case is marked as “Pass,” showing successful implementation of the functionality. This feature makes the queue joining process faster, easier, and more convenient for patients. Overall, the table demonstrates the successful testing and implementation of the QR code scanning feature in SwiftQueue.

Table 3.5: Receive Notifications

Field	Description
Test Case Id	UC-11
Test Case Name	Receive Notifications
Actor	Patient
Objective	To notify users about updates
Module	System
Input	Queue status change
Expected Result	Notification received
Actual Result	As expected
Status	Pass

This table describes the “Receive Notifications” test case of the SwiftQueue system. It explains how the system sends notifications to patients whenever there is a change in queue status or important update related to their appointment. The objective of this feature is to keep patients informed in real time and reduce unnecessary waiting at the clinic. The system module automatically triggers notifications based on queue activity and status changes. The successful test result confirms that patients receive notifications correctly and the functionality operates as expected without any errors.

Table 3.6: View Map Location

Field	Description
Test Case Id	UC-12
Test Case Name	View Map Location
Actor	Patient
Objective	To show clinic location
Module	Mobile App
Input	Open map
Expected Result	Location displayed
Actual Result	As expected
Status	Pass

Table 3.6 describes the “View Map Location” use case of the SwiftQueue system. This use case allows a patient to view the clinic location through the mobile application using map integration. The table explains the objective, module, input, and expected outcome of the functionality. The patient opens the map feature to locate the clinic easily and access navigation support. The expected result is that the clinic location is displayed successfully on the map. The actual result indicates that the system performed as expected during testing. The status of the test case is marked as “Pass,” which confirms the successful execution of the feature. This functionality helps patients reach clinics conveniently and reduces confusion in locating healthcare facilities. Overall, the table demonstrates the successful implementation and testing of the map location viewing feature in SwiftQueue.

Table 3.7: Track Queue in Real-Time

Field	Description
Test Case Id	UC-13
Test Case Name	Track Queue
Actor	Patient
Objective	To track queue status
Module	Mobile App
Input	Open tracking screen
Expected Result	Live queue position shown
Actual Result	As expected
Status	Pass

Table 3.7 describes the “Track Queue in Real-Time” use case of the SwiftQueue system. This use case allows a patient to monitor the live status and position of their queue through the mobile application. The table explains the objective, module, input, and expected outcome of the tracking functionality. The patient opens the tracking screen to view updated queue information in real time. The expected result is that the live queue position is displayed successfully on the screen. The actual result shows that the system performed as expected during testing. The status of the test case is marked as “Pass,” indicating successful implementation of the feature. This functionality helps patients stay informed about their turn and reduces unnecessary waiting at clinics. Overall, the table demonstrates the successful testing and implementation of the real-time queue tracking feature in SwiftQueue.

Table 3.8: View History

Field	Description
Test Case Id	UC-14
Test Case Name	View History
Actor	Patient
Objective	To view past records
Module	Mobile App
Input	Open history section
Expected Result	History displayed
Actual Result	As expected
Status	Pass

Table 3.8 describes the “View History” use case of the SwiftQueue system. This use case allows a patient to view previous records and past activities through the mobile application. The table explains the objective, module, input, and expected outcome of the history viewing functionality. The patient opens the history section to access details of previous appointments and queue records. The expected result is that the complete history is displayed successfully on the screen. The actual result indicates that the system performed as expected during testing. The status of the test case is marked as “Pass,” confirming successful implementation of the feature. This functionality helps patients maintain and review their healthcare interaction records conveniently. Overall, the table demonstrates the successful testing and implementation of the history viewing feature in SwiftQueue.

Table 3.9: Generate QR Code (Admin)

Field	Description
Test Case Id	UC-15
Test Case Name	Generate QR Code
Actor	Admin
Objective	To create QR for doctors
Module	Admin Panel
Input	Click generate QR
Expected Result	QR code generated
Actual Result	As expected
Status	Pass

Table 3.9 describes the “Generate QR Code” use case of the SwiftQueue system. This use case allows the admin to generate QR codes for doctors through the admin panel. The table explains the objective, module, input, and expected outcome of the QR code generation process. The admin selects the generate QR option to create a unique QR code for a doctor. The expected result is that the QR code is successfully generated and ready for use in queue joining. The actual result shows that the system performed as expected during testing. The status of the test case is marked as “Pass,” indicating successful implementation of the functionality. This feature helps simplify patient queue joining through QR scanning and improves overall system efficiency. Overall, the table demonstrates the successful testing and implementation of the QR code generation feature in SwiftQueue.

Table 3.10: Admin Login

Field	Description
Test Case Id	UC-16
Test Case Name	Admin Login
Actor	Admin
Objective	To authenticate admin
Module	Admin Panel
Input	Admin credentials
Expected Result	Admin logged in
Actual Result	As expected
Status	Pass

Table 3.10 describes the “Admin Login” use case of the SwiftQueue system. This use case allows the admin to securely access the admin panel by entering valid credentials. The table explains the objective, module, input, and expected outcome of the login process. The admin provides login credentials to authenticate access to the system. The expected result is that the admin is logged into the admin panel successfully. The actual result indicates that the system performed as expected during testing. The status of the test case is marked as “Pass,” confirming successful implementation of the functionality. This feature is important because it ensures secure access to administrative controls and system management features. Overall, the table demonstrates the successful testing and implementation of the admin login functionality in SwiftQueue.

Table 3.11: View Patients

Field	Description
Test Case Id	UC-17
Test Case Name	View Patients
Actor	Admin
Objective	To view patient records
Module	Admin Panel
Input	Open patient list
Expected Result	Patient data displayed
Actual Result	As expected
Status	Pass

Table 3.11 describes the “View Patients” use case of the SwiftQueue system. This use case allows the admin to view and manage patient records through the admin panel. The table explains the objective, module, input, and expected outcome of the patient viewing functionality. The admin opens the patient list to access patient details and records stored in the system. The expected result is that patient data is displayed successfully on the screen. The actual result indicates that the system performed as expected during testing. The status of the test case is marked as “Pass,” confirming successful implementation of the functionality. This feature helps the admin monitor patient information and maintain proper healthcare records efficiently. Overall, the table demonstrates the successful testing and implementation of the patient record viewing feature in SwiftQueue.

Table 3.12: Handle Patient Complaints

Field	Description
Test Case Id	UC-18
Test Case Name	Handle Complaints
Actor	Admin
Objective	To resolve patient issues
Module	Admin Panel
Input	Complaint submission
Expected Result	Complaint resolved
Actual Result	As expected
Status	Pass

Table 3.12 describes the “Handle Patient Complaints” use case of the SwiftQueue system. This use case allows the admin to review and resolve patient complaints through the admin panel. The table explains the objective, module, input, and expected outcome of the complaint handling functionality. The admin receives complaint submissions from patients and takes appropriate actions to resolve their issues. The expected result is that the complaint is handled and resolved successfully. The actual result shows that the system performed as expected during testing. The status of the test case is marked as “Pass,” confirming successful implementation of the functionality. This feature helps improve communication, patient satisfaction, and overall service quality within the healthcare system. Overall, the table demonstrates the successful testing and implementation of the patient complaint handling feature in SwiftQueue.

Table 3.13: Manage Queue

Field	Description
Test Case Id	UC-19
Test Case Name	Manage Queue
Actor	Admin
Objective	To control queue flow
Module	Admin Panel
Input	Update queue
Expected Result	Queue updated
Actual Result	As expected
Status	Pass

Table 3.13 describes the “Manage Queue” use case of the SwiftQueue system. This use case allows the admin to control and manage the queue flow through the admin panel. The table explains the objective, module, input, and expected outcome of the queue management functionality. The admin updates the queue to maintain proper patient flow and system organization. The expected result is that the queue is updated successfully within the system. The actual result shows that the system performed as expected during testing. The status of the test case is marked as “Pass,” confirming successful implementation of the functionality. This feature helps ensure smooth queue operations and improves efficiency in handling patients. Overall, the table demonstrates the successful testing and implementation of the queue management feature in SwiftQueue.

Table 3.14: Approve Doctors

Field	Description
Test Case Id	UC-20
Test Case Name	Approve Doctors
Actor	Admin
Objective	To verify doctors
Module	Admin Panel
Input	Approve request
Expected Result	Doctor approved
Actual Result	As expected
Status	Pass

Table 3.14 describes the “Approve Doctors” use case of the SwiftQueue system. This use case allows the admin to verify and approve doctor registration requests through the admin panel. The table explains the objective, module, input, and expected outcome of the doctor approval functionality. The admin reviews doctor requests and approves eligible doctors to access the system. The expected result is that the doctor is successfully approved after verification. The actual result indicates that the system performed as expected during testing. The status of the test case is marked as “Pass,” confirming successful implementation of the functionality. This feature helps maintain system security and ensures that only verified doctors can provide services through the platform. Overall, the table demonstrates the successful testing and implementation of the doctor approval feature in SwiftQueue.

3.3 Functional Requirements

3.3.1 User Registration & Authentication

The system allows users to register and log in securely using email and password. Authentication ensures that only authorized users can access the application.

3.3.2 Doctor Profile Management

Doctors can create and manage their profiles, including personal details and availability. This helps patients access accurate and updated information.

3.3.3 QR Code generation & Scanning

Each doctor is assigned a unique QR code for identification. Patients scan the QR code to quickly join the respective queue.

3.3.4 Queue Joining System

Patients can join queues digitally without physical waiting. The system assigns a position and updates it dynamically.

3.3.5 Appointment Slot Booking

The system allows patients to book available time slots in advance. This reduces congestion and improves scheduling efficiency.

3.3.6 Real-Time & Queue Tracking

Users can view their live position in the queue. The system updates queue status instantly using real-time data.

3.3.7 Notifications & Alerts

The system sends alerts to notify users about their upcoming turn. Notifications help reduce delays and improve user experience.

3.3.8 Payment History Management

The system stores records of past visits and appointments. Doctors and patients can access history for better decision-making.

3.3.9 Location & Map Integration

The system provides map-based location of clinics using GPS. Users can view directions and estimate travel time easily.

3.3.10 Admin Panel Management

The system includes a web-based admin panel developed using Laravel. The admin is responsible for managing system operations, including monitoring users, handling reports, and maintaining system integrity.

3.3.11 Doctor Verification System

Doctors must be verified before becoming active in the system.

- Newly registered doctors are marked as pending
- Admin reviews and approves/rejects
- Only approved doctors can generate QR codes

3.3.12 Complaint & Report Management

- Patients can report doctors in case of issues
- Reports are stored in database

Admin can:

- Mark as reviewed
- Resolve complaints
- Delete reports

3.4 Nonfunctional Requirements

3.4.1 Performance

The performance of the system is essential to ensure a smooth and efficient user experience. SwiftQueue is designed to respond quickly to user actions such as login, queue joining, and appointment booking. The system must handle multiple user requests simultaneously without slowing down or crashing. Real-time updates, such as queue position and notifications, should be delivered instantly to maintain accuracy and reliability. Efficient data handling and optimized backend processing help in minimizing delays and improving overall system responsiveness.

3.4.2 Security

Security is a critical requirement in SwiftQueue as it deals with sensitive user and healthcare-related data. The system ensures data protection through secure authentication mechanisms provided by Firebase Authentication. User information is stored safely in the

cloud with proper access control to prevent unauthorized usage. Encryption techniques are applied to protect data during transmission and storage. Additionally, the system restricts access based on user roles (patient, doctor, admin) to maintain privacy and system integrity.

3.4.3 Scalability

Scalability ensures that the system can handle growth in users and data without affecting performance. SwiftQueue is built using cloud-based services like Firebase, which allows dynamic scaling based on demand. As the number of patients, doctors, and transactions increases, the system can expand its capacity automatically. This ensures consistent performance even during peak usage times. The scalable architecture also makes it easier to integrate new features and support future system enhancements.

3.4.4 Accessibility

Accessibility focuses on making the system usable for a wide range of users, regardless of their technical expertise. SwiftQueue is designed with a simple and clean user interface that is easy to understand and navigate. The application ensures that patients, doctors, and administrators can access features without confusion. Clear labels, organized layouts, and minimal complexity improve usability for all users. This approach helps in making the system inclusive and practical for real-world healthcare environments.

3.4.5 Usability

Usability ensures that the system provides a convenient and user-friendly experience. SwiftQueue is designed with intuitive navigation and a well-structured interface to reduce user effort. Features such as QR scanning, queue tracking, and notifications are implemented in a simple and straightforward manner. The system minimizes unnecessary steps, allowing users to complete tasks quickly and efficiently. Overall, good usability enhances user satisfaction and encourages regular use of the application.

Chapter 4

Design and Architecture

4. Design and Architecture

The design and architecture phase plays a crucial role in transforming system requirements into a structured and functional blueprint. It defines how different components of the system interact with each other and ensures that the system is scalable, maintainable, and efficient. A well-designed architecture not only improves system performance but also makes future enhancements easier to implement. In the case of SwiftQueue, careful planning was carried out to ensure that both the mobile application and the web-based admin panel work seamlessly together while maintaining high performance and reliability.

For SwiftQueue, the architecture is designed to support real-time operations, seamless user interaction, and secure data handling. The system follows a client-server architecture, where the mobile application acts as the client and Firebase serves as the backend. The client side is responsible for handling user interactions, displaying information, and sending requests, while the backend processes these requests, manages data, and ensures synchronization across all users. This separation of concerns improves system organization and allows independent development and maintenance of different components.

The mobile application, developed using Flutter, provides an intuitive and responsive interface for patients. It allows users to perform various actions such as registering, logging in, scanning QR codes, joining queues, tracking queue status in real time, receiving notifications, and viewing location details through map integration. The user interface is designed with simplicity and usability in mind, ensuring that users with different levels of technical knowledge can easily interact with the system. On the other hand, the web-based admin panel provides a centralized control system for doctors and administrators. It allows them to manage queues, approve doctors, view patient details, handle complaints, and monitor overall system activity efficiently.

Firebase plays a central role in the architecture by providing cloud-based services such as Firestore database, authentication, and real-time synchronization. Firestore is used to store and manage all system data, including user information, queue details, appointments, and notifications. The real-time capabilities of Firebase ensure that any change in the system, such as queue updates or appointment status, is instantly reflected across all connected devices. Firebase Authentication provides a secure mechanism for user login and access control, ensuring that only authorized users can access the system.

In addition to Firebase, external services such as Google Maps API are integrated to provide location-based functionalities. This allows patients to view hospital locations and navigate easily, enhancing the overall user experience. The system also uses push notification services to keep users informed about their queue status, appointment reminders, and important updates. This real-time communication improves coordination between patients and healthcare providers.

The architecture is designed with scalability in mind, allowing the system to handle an increasing number of users without performance degradation. Since the backend is cloud-based, it can easily scale resources according to demand. Security is also a key consideration, with proper authentication, data validation, and controlled access implemented to protect user information. Furthermore, the modular design of the system ensures that new features can be added in the future without affecting existing functionalities.

This architecture ensures that data is processed efficiently and updates are delivered instantly to all users. It provides a reliable foundation for the SwiftQueue system, enabling smooth communication between the mobile application and the web panel. Overall, the design and architecture of SwiftQueue support real-time operations, enhance user experience, and ensure the system remains robust, flexible, and future-ready.

4.1 System Architecture

The overall system architecture of the SwiftQueue healthcare queue management system. The system is divided into three main components: the frontend (mobile application and web admin panel), the backend server, and external services (Firebase and Google Maps API).

On the left side, users including patients, doctors, and administrators interact with the system through the frontend. The mobile application provides functionalities such as joining queues, logging in, scanning QR codes, viewing map locations, receiving notifications, and tracking queues in real time. The web-based admin panel allows administrators and doctors to manage system operations, including viewing patients, managing queues, approving doctors, generating QR codes, and handling complaints.

The frontend communicates with the backend (Laravel server), which processes requests and manages system logic. The backend is integrated with Firebase services, including Firebase Firestore for data storage, Firebase Authentication for user verification, and Firebase Cloud Messaging for real-time notifications.

Additionally, the system integrates the Google Maps API to provide location-based services, enabling users to view clinic locations easily.

Overall, this architecture ensures a scalable, real-time, and efficient system by combining mobile and web interfaces with cloud-based services for data management and communication.

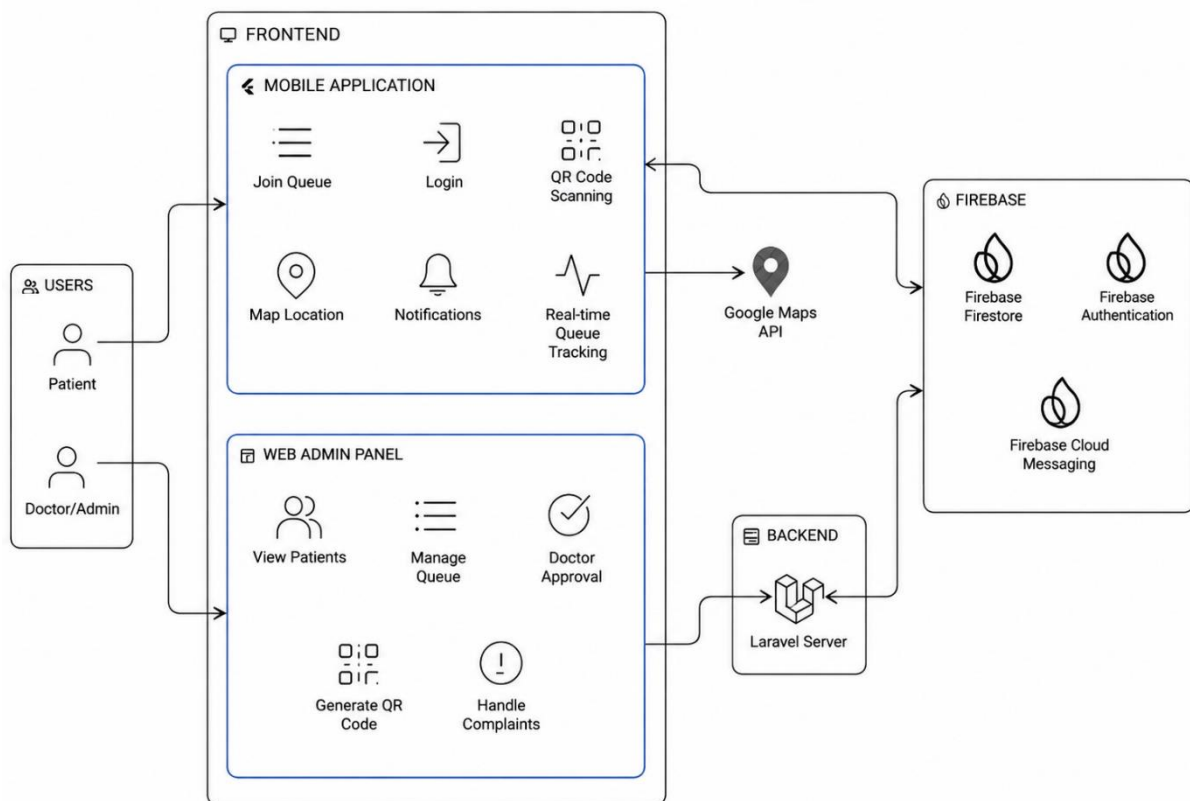


Figure 4.1

The figure illustrates the overall system architecture of the SwiftQueue healthcare queue management system. It shows the interaction between users, the frontend interfaces, backend server, Google Maps API, and Firebase services. Patients use the mobile application to perform activities such as joining queues, scanning QR codes, tracking queues in real time, viewing map locations, and receiving notifications. The admin accesses the web admin panel to manage queues, approve doctors, generate QR codes, view patient records, and handle complaints. The diagram also highlights the integration of Laravel as the backend server and Firebase services for authentication, real-time database management, and cloud messaging. Overall, the figure demonstrates how different components of SwiftQueue work together to provide an efficient, real-time, and user-friendly healthcare queue management system.

4.2 Process Flow / Representation

The working of SwiftQueue follows a structured flow:

1. The user launches the application and logs in using secure credentials.
2. The patient scans the QR code provided by the doctor or clinic.
3. The system validates the QR code and retrieves the doctor's queue information.
4. The patient is added to the queue and assigned a position.
5. The queue is updated dynamically as patients are served.
6. The system continuously updates the patient's position in real time.
7. When a doctor registers, the admin verifies the doctor's license before granting access to the dashboard.

This process ensures a smooth and efficient workflow, reducing waiting time and improving user experience.

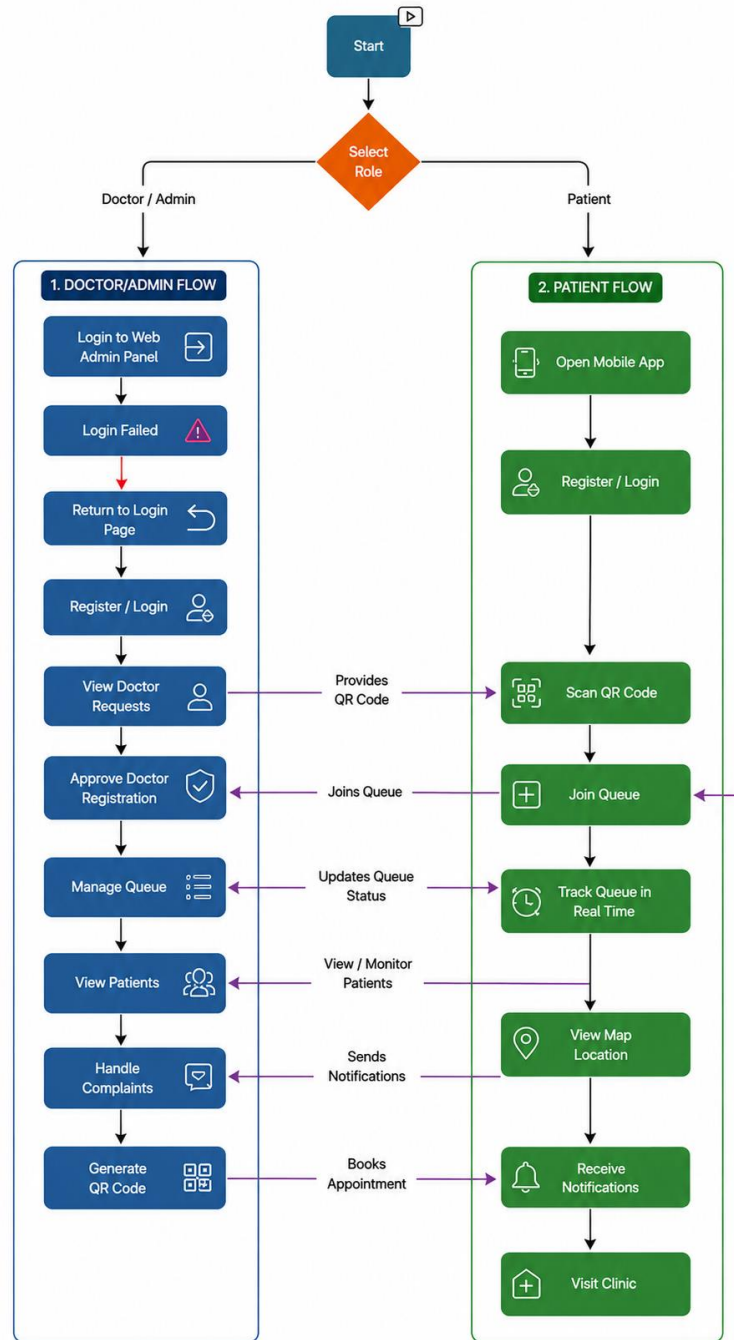


Figure 4.2

The figure illustrates the complete workflow of the SwiftQueue system for both Doctor/Admin and Patient users. It begins with role selection, where users choose either the Doctor/Admin flow or the Patient flow according to their role in the system. The Doctor/Admin side includes functionalities such as login, registration, viewing doctor requests, approving doctors, managing queues, viewing patients, handling complaints, generating QR codes, and

handling login failures through a return-to-login process. The Patient side demonstrates features such as opening the mobile application, registering or logging in, scanning QR codes, joining queues, tracking queue status in real time, viewing clinic locations, receiving notifications, and visiting clinics. The diagram also shows the interaction between both flows through actions like queue joining, queue updates, notifications, and appointment handling. Overall, the figure explains the operational flow and interaction process of the SwiftQueue healthcare queue management system in a clear and organized manner.

4.3 Sequence Diagram

The sequence diagram illustrates the interaction between different components of the system over time. It shows how requests are sent and responses are received between the user, application, and database.

For example, when a patient scans a QR code:

- The request is sent from the mobile app to Firebase.
- Firebase verifies the QR code and retrieves queue data.
- The system assigns a queue position to the patient.
- The updated queue information is sent back to the application.

This sequence ensures that all operations are performed in a logical and synchronized manner.

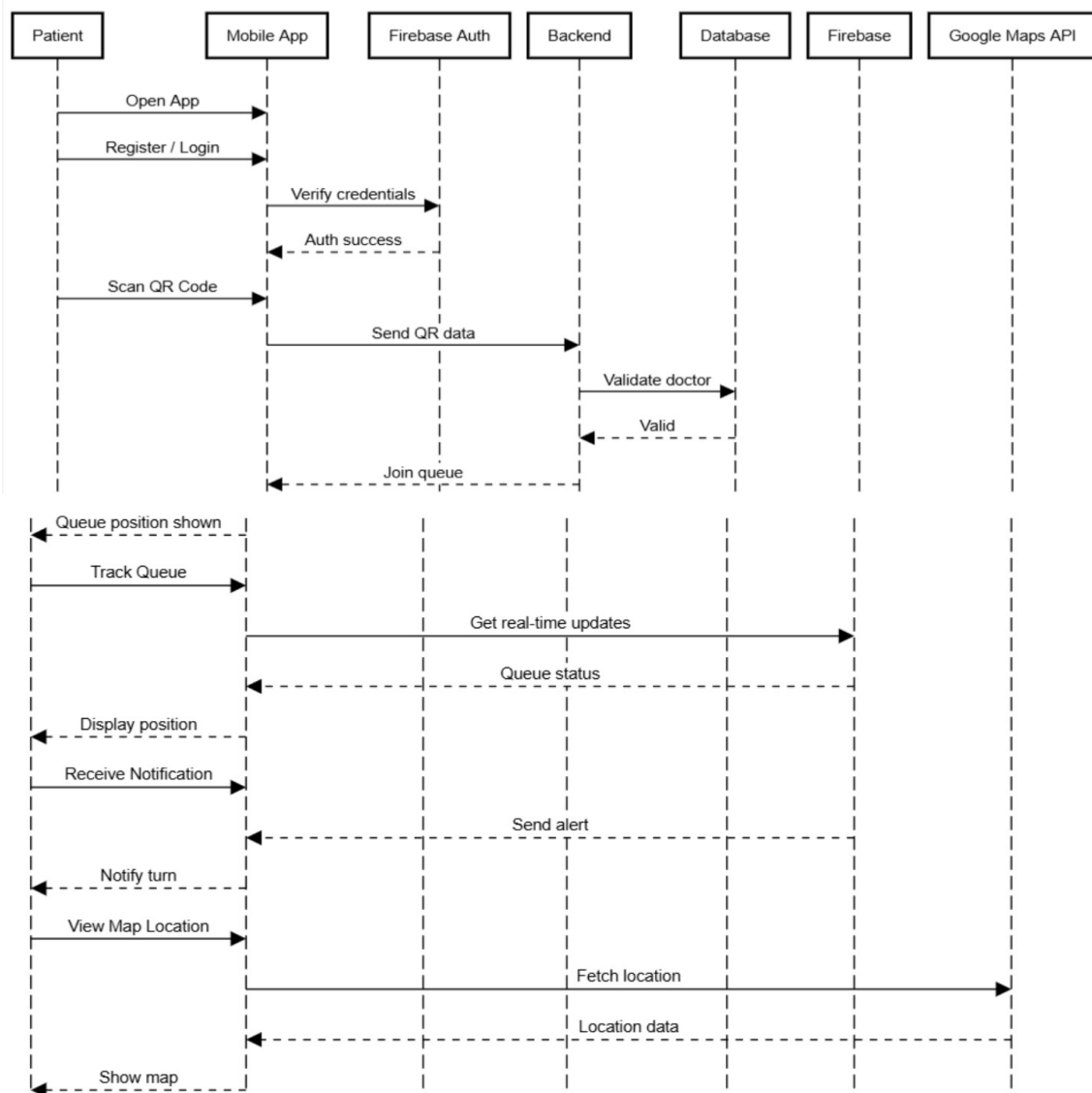


Figure 4.3 Patient Sequence Diagram

The figure illustrates the patient sequence diagram of the SwiftQueue system. It shows the interaction between the patient, mobile application, Firebase Authentication, backend server, database, Firebase, and Google Maps API during different system operations. The sequence begins with the patient opening the mobile app, registering or logging in, and scanning a QR code to join a queue. The diagram also demonstrates real-time queue tracking, notification alerts, and viewing clinic locations through Google Maps integration. Additionally, it highlights the flow of data validation, queue updates, and communication between system components. Overall, the figure explains how different technologies and modules work together to provide a smooth and real-time patient experience in SwiftQueue.

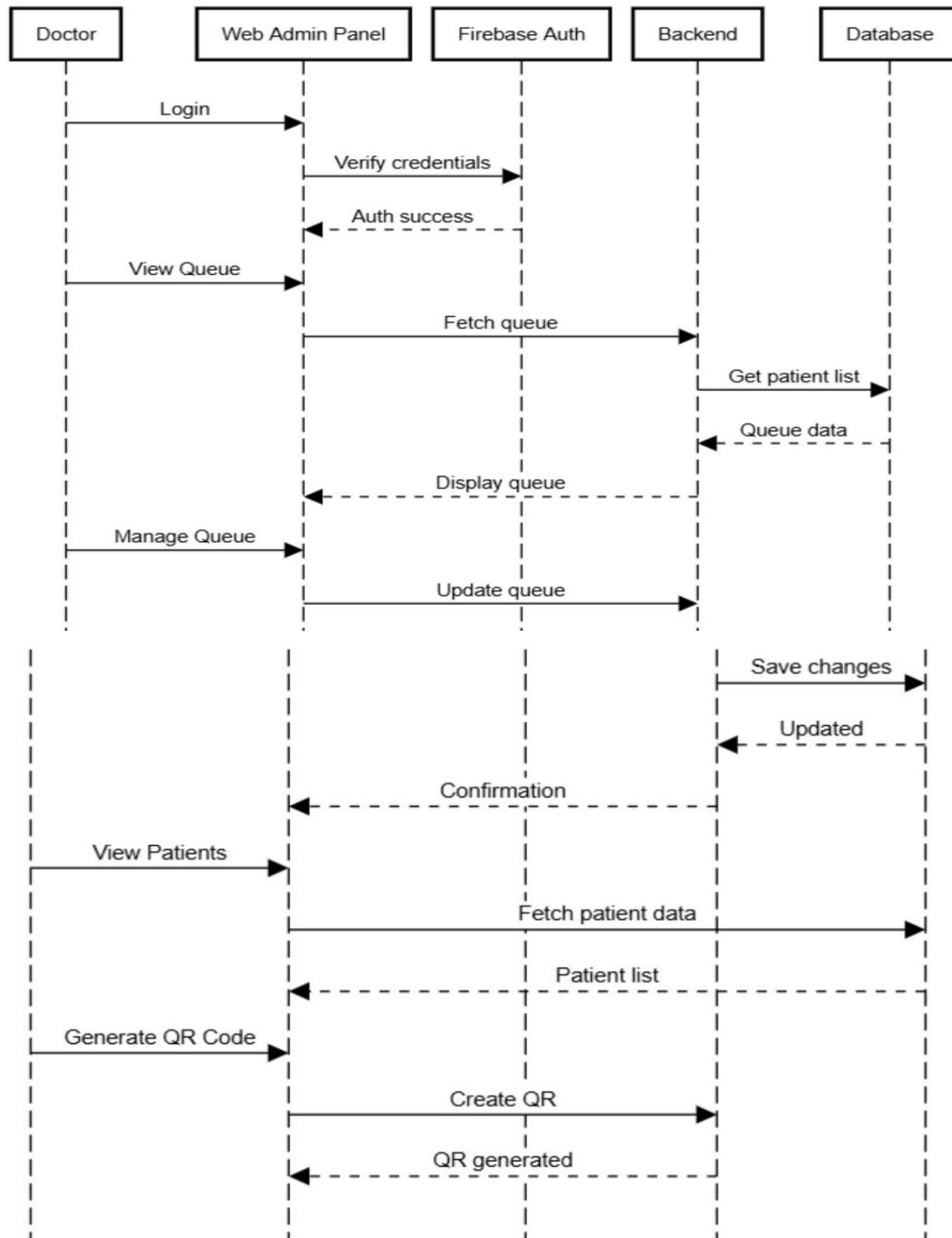


Figure 4.3 Doctor Sequence Diagram

The figure presents the complete workflow and interaction process of the SwiftQueue healthcare management system. It demonstrates how patients, doctors, admins, mobile applications, backend services, Firebase, databases, and Google Maps API communicate with each other to perform operations such as login, queue management, QR scan ing, notifications, and appointment handling. Overall, the diagram explains the integrated architecture and real-time functioning of the SwiftQueue system in an organized and systematic manner.

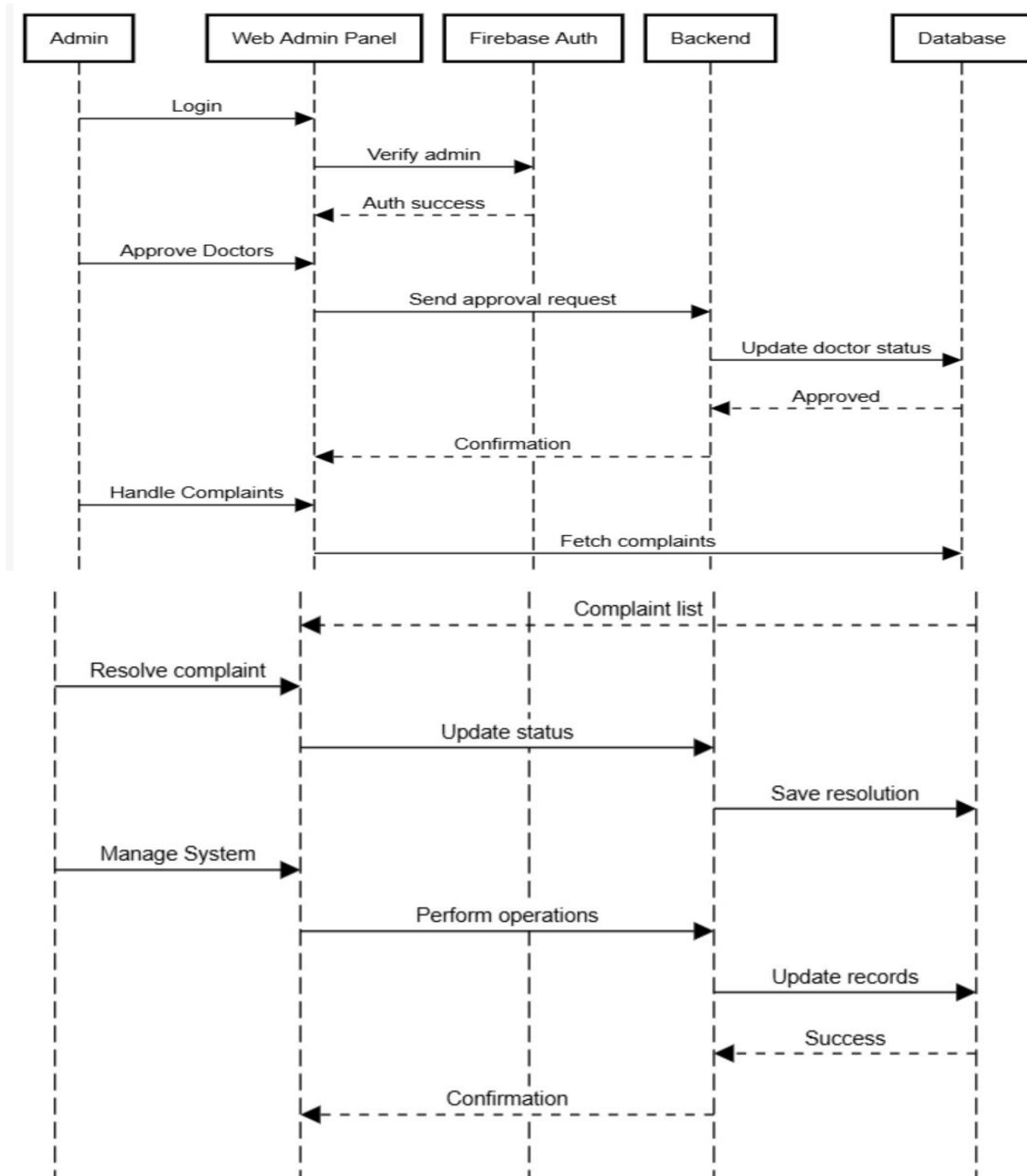


Figure 4.3 Admin Sequence Diagram

The figure illustrates the admin sequence diagram of the SwiftQueue system. It explains the interaction between the admin, web admin panel, Firebase Authentication, backend server, and database during different administrative operations. The diagram shows processes such as admin login, doctor approval, complaint handling, complaint resolution, and overall system management with database updates and confirmations. Overall, the figure demonstrates how the admin panel communicates with backend services and the database to manage and control the healthcare queue management system efficiently.

4.4 Class Diagram

The class diagram represents the structure of the system by defining its classes, attributes, and relationships.

Key classes in SwiftQueue include:

- **User Class** → Stores user details such as name, email, and role
- **Doctor Class** → Contains doctor-specific information
- **Queue Class** → Manages queue data and positions
- **Appointment Class** → Handles booking details

The relationships between these classes ensure smooth data flow and system functionality.

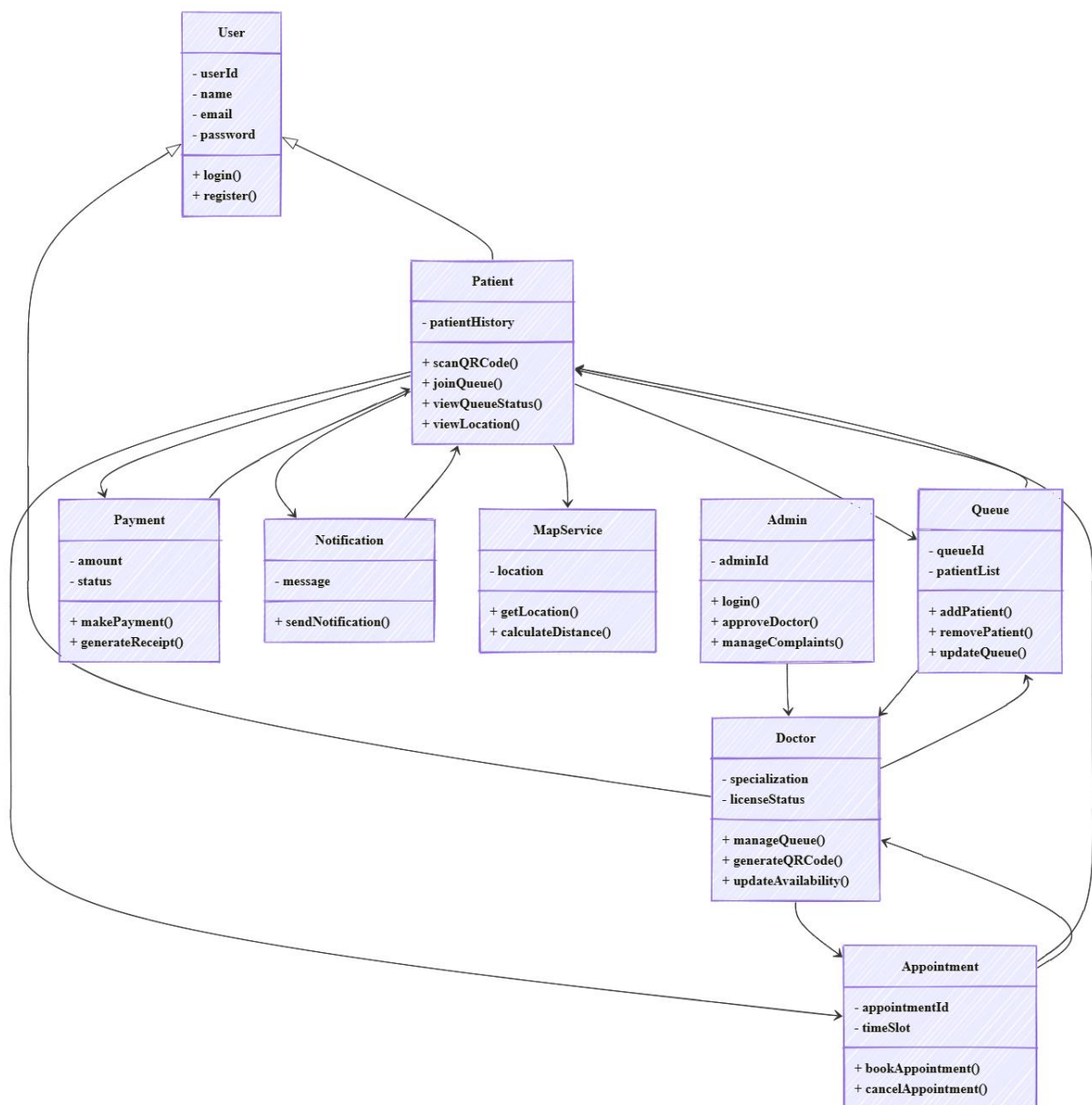


Figure 4.4

The figure illustrates the class diagram of the SwiftQueue healthcare queue management system. It represents the main classes of the system, including User, Patient, Doctor, Admin, Queue, Appointment, Notification, Payment, and MapService, along with their attributes and functions. The diagram also shows the relationships and interactions between different classes, such as queue management, appointment booking, QR code scanning, notifications, and payment handling. Additionally, it explains how doctors, patients, and admins perform different operations within the system while communicating with related services. Overall, the figure provides a structured overview of the object-oriented design and functionality of the SwiftQueue system.

4.5 ER Diagram

This ER diagram represents the overall database structure of the SwiftQueue system. It shows the key entities and their relationships used to manage users, queues, appointments, and system operations.

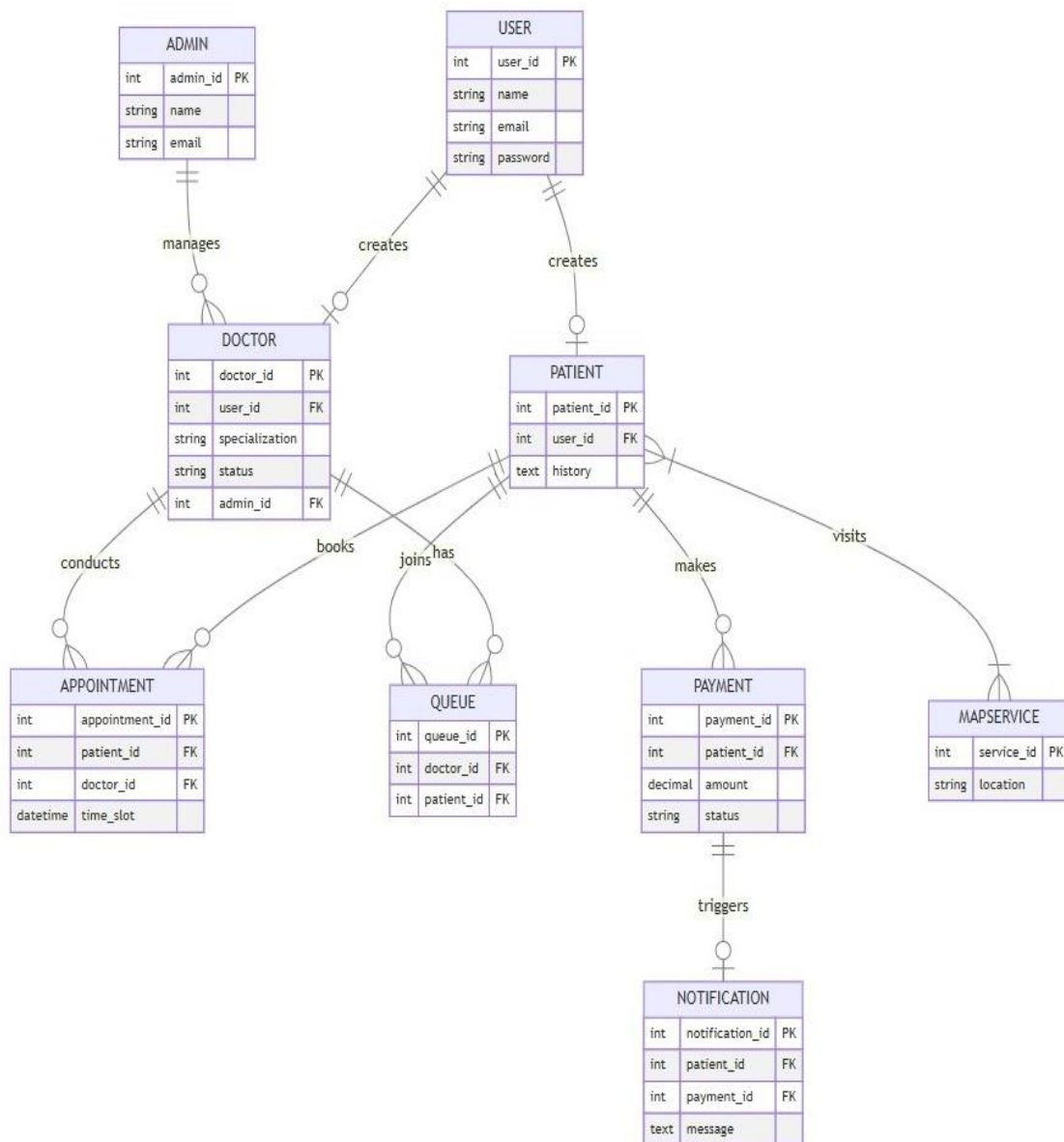


Figure 4.5

This image presents the Entity Relationship (ER) Diagram of the SwiftQueue system, illustrating the structure of the database and relationships between different entities. It includes key entities such as User, Admin, Doctor, Patient, Appointment, Queue, Payment, Notification, and Map Service. The diagram shows how users create doctor and patient profiles, while admins manage doctors and system operations. It also highlights interactions like patients booking appointments, joining queues, and making payments. Overall, the diagram represents how data flows and is organized to support efficient healthcare queue management.

Chapter 5

Implementation

5. Implementation

The implementation phase involves converting the system design into a working application. SwiftQueue is developed using Flutter for the mobile frontend, a web-based admin panel for system control, and Firebase services for real-time data handling and authentication. This combination of technologies ensures a modern, efficient, and responsive system that meets user requirements.

The mobile application is designed for patients and provides features such as registration/login, QR code scanning, queue joining, real-time queue tracking, notifications, and map integration. Flutter enables a smooth and visually appealing user interface, ensuring that the application performs well across different devices while maintaining consistency in design and functionality.

The web-based admin panel is developed to manage system operations, including doctor approval, queue management, patient handling, complaint management, and QR code generation. This panel allows administrators and doctors to control system activities efficiently. The separation between mobile and web components ensures better organization and easier maintenance of the system.

Firebase serves as the backend, providing Firestore for data storage, Authentication for secure user access, and Cloud Messaging for real-time notifications. The system is implemented in a modular manner, allowing independent development and testing of each component while ensuring seamless communication between them. This approach enhances scalability, reliability, and overall system performance.

5.1 User Registration Algorithm

The user registration process ensures secure account creation and authentication.

Steps involved:

1. User enters email and password
2. System validates input
3. Firebase Authentication creates user account
4. User data is stored in Firestore
5. User is redirected to dashboard

This process ensures secure and efficient user onboarding.

5.2 QR Code Generation Algorithm

Each doctor is assigned a unique QR code.

Steps:

1. Generate unique doctor ID
2. Convert ID into QR code
3. Store QR data in database
4. Display QR code on doctor dashboard

This allows patients to join queues quickly and accurately.

5.3 Queue Management Algorithm

The queue system dynamically updates positions.

Steps:

1. Patient joins queue
2. System assigns position
3. When a patient is served, queue shifts
4. All positions are updated automatically

This ensures fairness and efficiency.

5.4 Appointment Scheduling Logic

The system allows patients to book appointments in advance.

Steps:

1. Patient selects available slot
2. System checks availability
3. Slot is reserved
4. Data is stored in database

This reduces congestion and improves time management.

5.5 Real-Time Queue Tracking System

This feature is the core of SwiftQueue.

- Firebase streams are used for real-time updates
- Any change in queue is instantly reflected
- Patients can see live position

5.6 Security and Data Protection

Security is implemented using:

- Firebase Authentication
- Encrypted data storage
- Secure API communication

5.7 User Interface

The interface is designed using Flutter widgets.

5.7.1 Welcome Screen

Introduces application features

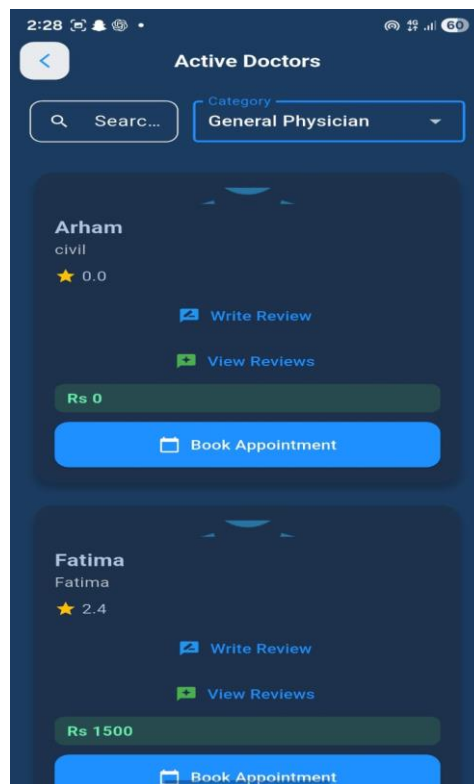


Figure 5.8

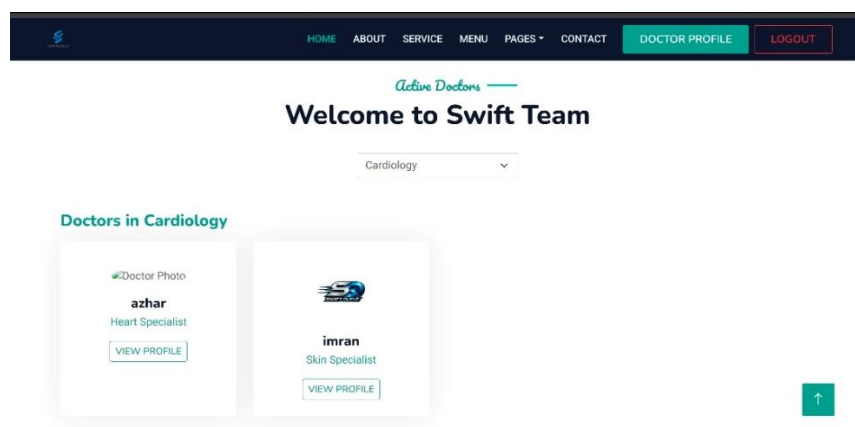
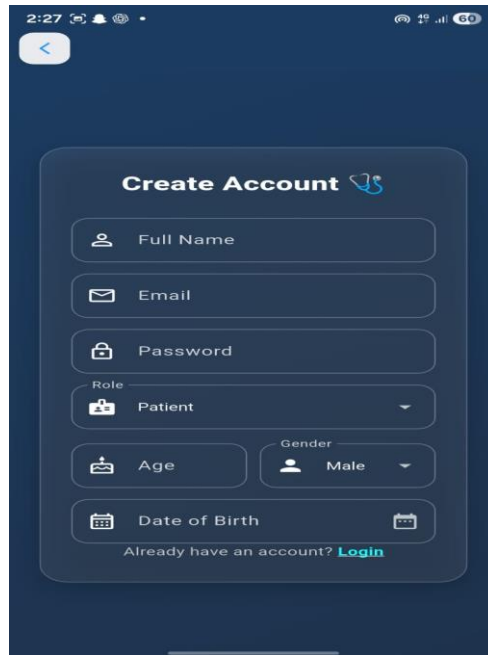


Figure 5.8

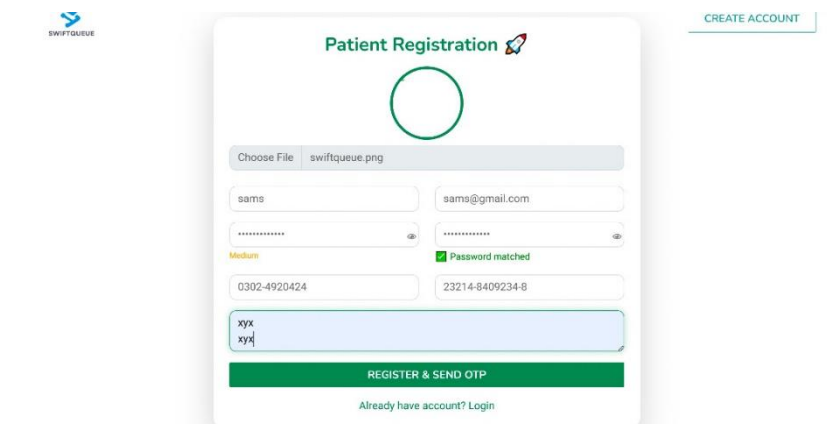
5.7.2 Login Screen

Handles authentication



A mobile application screen titled "Create Account" with a dark blue background. It features a central white rounded rectangle containing several input fields: "Full Name", "Email", "Password", "Role" (with a dropdown menu showing "Patient"), "Age", "Gender" (with a dropdown menu showing "Male"), and "Date of Birth" (with a calendar icon). At the bottom of the white area, it says "Already have an account? [Login](#)". The top of the screen shows a status bar with the time 2:27 and various icons.

Figure 5.9



A web application form titled "Patient Registration" with a white background. It includes a profile picture placeholder (a green circle) and a file upload section labeled "Choose File" with the filename "swiftqueue.png". Below this are input fields for "Name" (containing "sams"), "Email" (containing "sams@gmail.com"), "Password" (with a strength indicator "Medium"), and "Confirm Password" (containing "xyx"). A green checkmark and the text "Password matched" are visible between the password fields. There are also fields for "Phone Number" (containing "0302-4920424") and "OTP" (containing "23214-8409234-8"). At the bottom, there is a green button labeled "REGISTER & SEND OTP" and a link "Already have account? [Login](#)". The SwiftQueue logo is in the top left, and a "CREATE ACCOUNT" link is in the top right.

Figure 5.9

5.7.3 Patient Dashboard

Displays queue status and options

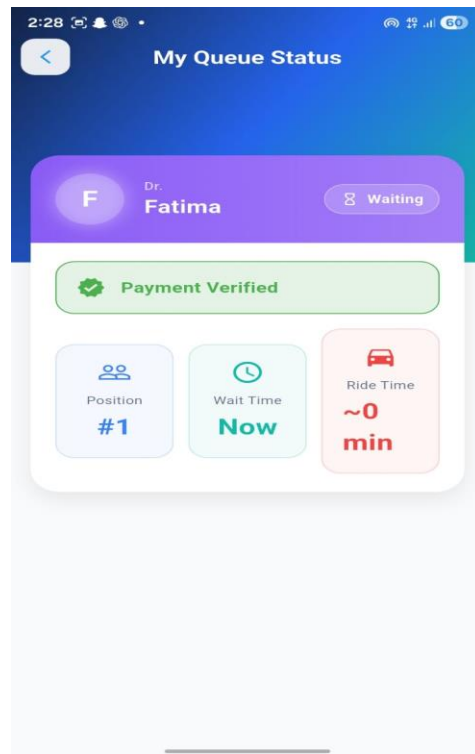


Figure 5.7.3

5.7.4 Doctor Dashboard

Shows patient queue

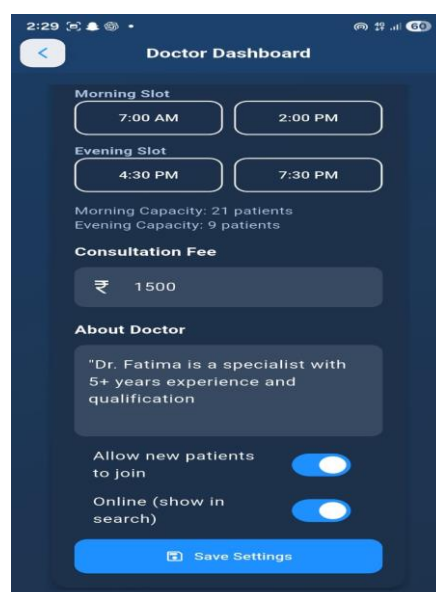


Figure 5.7.4

5.7.5 QR Scan Screen

Allows joining queue

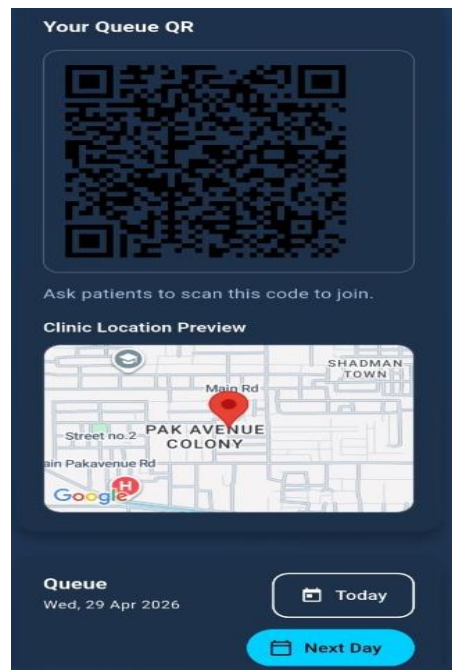


Figure 5.7.5

5.7.6 Queue Status Screen

Shows live updates

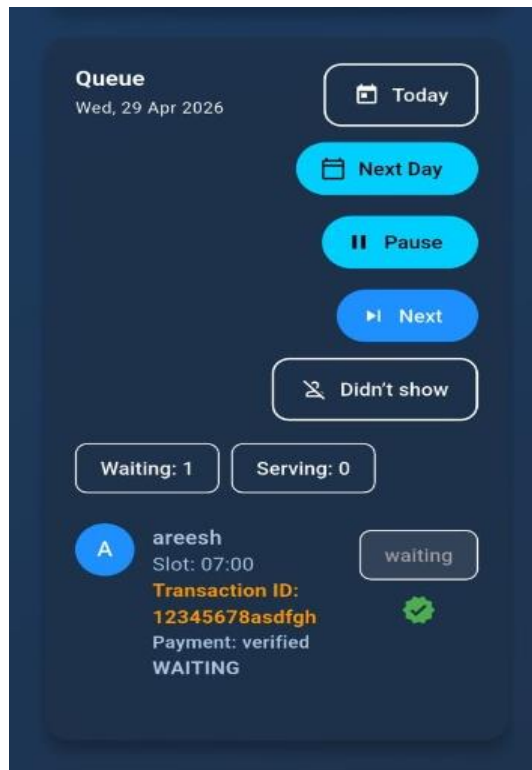


Figure 5.7.6

5.8 Smart Queue Integration System

SwiftQueue calculates estimated waiting time using previous service durations stored in Fire-base. The system analyzes the average consultation time and dynamically updates the expected waiting time for each patient. This helps users plan their arrival efficiently and reduces unnecessary waiting.

Chapter 6

Testing and Evaluation

6. Testing

Testing ensures that the system functions correctly and meets user requirements effectively. It is performed at multiple levels to identify, analyze, and fix errors during development. Different testing techniques such as unit testing, integration testing, and system testing are applied to verify each component of the system. This process helps in ensuring that all modules work smoothly together without any issues. Overall, testing improves system reliability, performance, and user satisfaction.

6.1 Unit Testing

Unit testing focuses on individual components such as login, QR scanning, and queue management. Each module is tested independently to ensure correctness.

Table 6.1: Unit Testing

Test Case Id	UT-01
Test Case Name	User Authentication Testing
Objective	To verify login and signup functionality
Module	Mobile App / Admin Panel
Input	Valid & invalid credentials
Expected Result	Successful login for valid data, error for invalid data
Actual Result	As expected
Status	Pass

This table presents the unit testing details for the User Authentication module of the SwiftQueue system. It verifies the functionality of login and signup processes using both valid and invalid credentials. The objective is to ensure that the system correctly authenticates users and handles errors appropriately. The expected results confirm successful login for valid inputs and error messages for invalid ones. The actual result matches the expectation, indicating that the test case has successfully passed.

6.2 Manual Testing

Manual testing involves checking the system without automated tools. The application is tested for usability, navigation, and user experience.

Table 6.2: Manual Testing

Test Case Id	MT-01
Test Case Name	QR Code Scanning
Objective	To check if QR scanning works correctly
Module	Mobile App
Input	Doctor QR Code
Expected Result	Patient joins queue successfully
Actual Result	As expected
Status	Pass

This table presents the manual testing details for the QR Code Scanning functionality in the SwiftQueue system. It focuses on verifying whether the QR scanning feature in the mobile application works correctly. The test uses a doctor's QR code as input to check if the patient can successfully join the queue. The expected and actual results match, confirming that the feature performs as intended. The successful outcome indicates that the test case has passed and the functionality is reliable.

6.3 Functional Testing

Functional testing verifies that all features work as expected. This includes:

- Registration
- Queue joining
- Notifications

Table 6.3: Functional Testing

Test Case Id	FT-01
Test Case Name	Queue Joining System
Objective	To validate queue joining functionality
Module	Mobile App
Input	Scan QR → Join Queue
Expected Result	Queue position assigned
Actual Result	As expected
Status	Pass

This table represents functional testing of the Queue Joining System in SwiftQueue. It verifies that scanning a QR code successfully assigns a queue position, with results matching the expected outcome.

6.4 Integration Testing

Integration testing ensures that different modules work together seamlessly. For example, the QR scanning module must correctly interact with the queue system.

Table 6.4: Integration Testing

Test Case Id	IT-01
Test Case Name	Firebase & Xamp Integration
Objective	To test communication between app and backend
Module	Mobile App + Admin Panel
Input	Data sync requests
Expected Result	Real-time updates reflected correctly
Actual Result	As expected
Status	Pass

This table shows integration testing for the Firebase and xamp integration in SwiftQueue. It verifies that the mobile app and admin panel communicate correctly with the backend through data synchronization. The successful result confirms real-time updates are working properly.

6.5 System Testing

System testing evaluates the complete application. It ensures that:

- The system performs well under load
- Data is handled correctly
- All features function together

Table 6.5: System Testing

Test Case Id	ST-01
Test Case Name	Complete System Workflow
Objective	To test full system from login to queue completion
Module	Entire System
Input	Login → Scan QR → Track Queue
Expected Result	Smooth execution without errors
Actual Result	As expected
Status	Pass

This table presents system testing for the **complete SwiftQueue workflow**, covering all major functionalities. It verifies the full process from login to QR scanning and queue tracking. The expected result is smooth execution without errors across the entire system. The successful outcome confirms that the system works reliably as a whole.

6.6 Testing Summary

The SwiftQueue system was tested under multiple scenarios to evaluate its performance, reliability, and overall functionality. Different modules including login, registration, QR code scanning, queue management, appointment booking, notifications, and real-time queue tracking were tested successfully. The system handled multiple users simultaneously without major delays or performance issues. Real-time updates were accurately reflected across both the mobile application and web admin panel. Authentication and database operations functioned correctly with secure data handling. The QR code generation and scanning features worked efficiently for queue joining and verification. Notification services provided timely alerts to patients regarding queue status and appointments. The admin panel successfully managed doctor approvals, complaints, patient records, and queue operations. No critical system failures or security issues were observed during the testing process. Overall, the testing results confirmed that SwiftQueue is a reliable, stable, and efficient healthcare queue management system.

Chapter 7

Conclusion and Future Work

7 Conclusion and Future Work

SwiftQueue successfully transforms traditional queue management into a smart digital system by integrating real-time tracking, QR-based access, and cloud-based services. The system not only improves efficiency but also enhances patient satisfaction and reduces operational burden on healthcare staff. Its scalable architecture makes it suitable for real-world deployment across multiple healthcare environments.

7.1 Conclusion

SwiftQueue successfully addresses the challenges faced in traditional healthcare queue management systems by introducing a smart, digital, and real-time solution. The system was developed to reduce patient waiting time, improve queue transparency, and enhance communication between patients, doctors, and administrators. Through the use of modern technologies such as Flutter, Firebase, Laravel, and Google Maps API, the project provides an efficient platform for managing healthcare queues digitally. The mobile application allows patients to register, log in, scan QR codes, join queues, track their queue status in real time, receive notifications, and view clinic locations easily. These features improve patient convenience and reduce overcrowding in healthcare facilities.

The admin panel provides centralized management functionalities including doctor approval, queue management, complaint handling, patient monitoring, and QR code generation. This enables administrators to control healthcare operations effectively and maintain organized patient flow. The QR code functionality simplifies the queue joining process and minimizes manual errors during patient registration and queue handling. Real-time queue updates ensure that patients receive accurate information about their turn and expected waiting time. The notification system further improves communication by informing patients about queue progress and appointment updates instantly.

The integration of Google Maps API helps patients locate clinics and healthcare facilities without difficulty. Firebase Authentication ensures secure login and registration processes, while Firestore supports reliable real-time data synchronization across the system. Laravel backend integration strengthens system performance and administrative control. The project also focused on providing a user-friendly interface to ensure ease of use for patients, doctors, and administrators. The testing phase confirmed that all major functionalities worked correctly under different scenarios without major failures.

SwiftQueue improves healthcare efficiency by reducing manual workload and enhancing operational management. It contributes to better patient satisfaction by saving time and providing transparency in healthcare services. The system architecture was designed to be scalable and flexible, making it suitable for implementation in different healthcare environments such as hospitals, clinics, and medical centers. The project successfully demonstrates how modern digital technologies can solve real-world healthcare management problems effectively.

Overall, SwiftQueue achieved all its major objectives and proved to be a reliable, secure, and efficient healthcare queue management system. It improves patient experience, supports healthcare staff, and streamlines queue operations through automation and real-time communication. The project highlights the importance of digital transformation in healthcare services and provides a practical solution for modern healthcare institutions. In conclusion, SwiftQueue can serve as a scalable and effective platform for improving healthcare queue management and delivering better healthcare services in the future

7.2 Future Work

The system can be further improved by adding advanced features and expanding its functionality

7.2.1 Real-time Queue Prediction

In future work, SwiftQueue can be improved by integrating AI-based real-time queue prediction features. The system can analyze patient flow, doctor availability, and historical queue data to predict waiting times more accurately. This enhancement would help patients receive estimated turn timings and suggested visiting schedules, further reducing overcrowding and improving healthcare efficiency.

7.2.2 Multi-Hospital Integration

The system can be extended to support multiple hospitals and clinics on a single platform. This will allow users to access different healthcare services from one application.

7.2.3 Xamp and Firebase Integration

The system can be further enhanced by integrating Xamp with Firebase to create a more robust and scalable backend architecture. Xamp can be used for handling complex server-side operations, API management, and admin panel functionalities, while Firebase can continue to provide real-time database updates, authentication, and cloud services.

This integration will combine the strengths of both technologies, allowing efficient data handling, improved performance, and better system scalability. It will also enable advanced features such as secure API communication, centralized control, and enhanced real-time synchronization between the mobile application and the web-based admin panel.

7.3 Limitations of the system

- Requires internet connectivity.
- Depends on smartphone availability.
- Initial setup required for clinics

References

<https://sequencediagram.org/>

<https://app.eraser.io/workspace/C2a3fbfrGfStqyUcQEqn>

<https://www.eraser.io/ai/flowchart-generator>

<https://chat.z.ai/c/71aac7f6-30a7-4d0b-992c-bfa9360e121a>

<https://evernote.com/ai-diagrams/class-diagram-generator-for-software-engineers>